Toward Reliable Detection of Android Lifecycle Violations with Static Analysis and Domain-Adapted Code Models

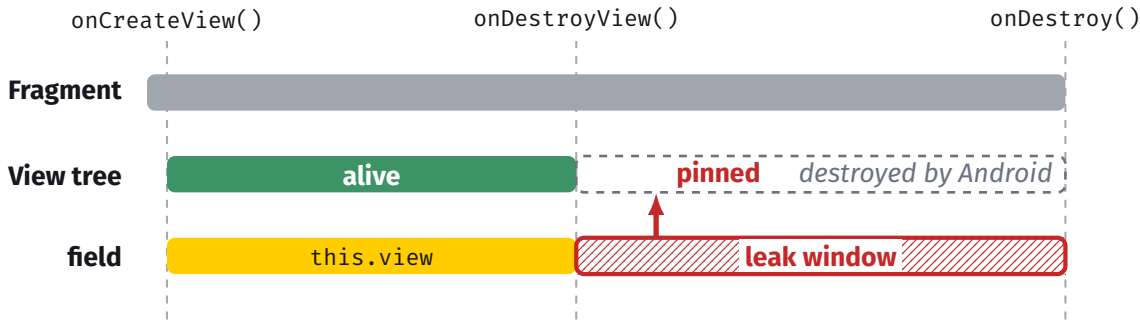
Muhammad Tehmasib Ali Tashfeen

Advisor: Dr. Zhiyong Shan · School of Computing, Wichita State University

ISSRE 2026 Doctoral Symposium



The View dies; the reference lives on



Memory
dead View tree + Activity Context

Stability
OOM after repeated transactions

Silent
compiles, passes tests, ships

One missing line leaks a whole View tree


```
class MusicianFragment extends Fragment {
  private View view;                                  retained

  View onCreateView(LayoutInflater inf, ...) {
    view = inf.inflate(...);
    return view;
  }
+ void onDestroyView() { ...; view = null; }
}
```

+1

line fixes it

Pattern of a real LeakScope finding in an open-source app (simplified).

Same root cause, many shapes

Fragment
View field never cleared

```
private FragmentBinding binding;   // never nulled
```

Thread
Worker captures UI

```
executor.execute(() -> activity.render(data));
```

ViewModel
State outlives a Context

```
class FeedVM extends ViewModel { Context ctx; }
```

Callback
Never unregistered

```
registerReceiver(rx, filter);      // no unregister
```

Common shape
framework-owned component → *retained* reference → *missing* cleanup

General tools see only part of it

Android Lint

+ framework-aware

– few lifecycle cleanup rules

106

of 627 apps flagged

SpotBugs

+ mature Java patterns

– not Android-lifecycle aware

344

of 627 apps flagged

Infer

+ deep interpro-cedural

– no lifecycle leak checks

0

of 627 apps flagged

Dynamic

(e.g. LeakCanary)

+ real heap evi-dence

– only exercised paths

—

needs execution

Lifecycle obligations are **contextual**: ownership, boundaries, cleanup.

Thesis: rules and models, together

Reliable lifecycle detection needs

traceable static evidence *and* **domain-adapted contextual reasoning**.

Static rules

+ explainable, auditable
– brittle to context

Code LLMs

+ read surrounding context
– opaque, need domain data

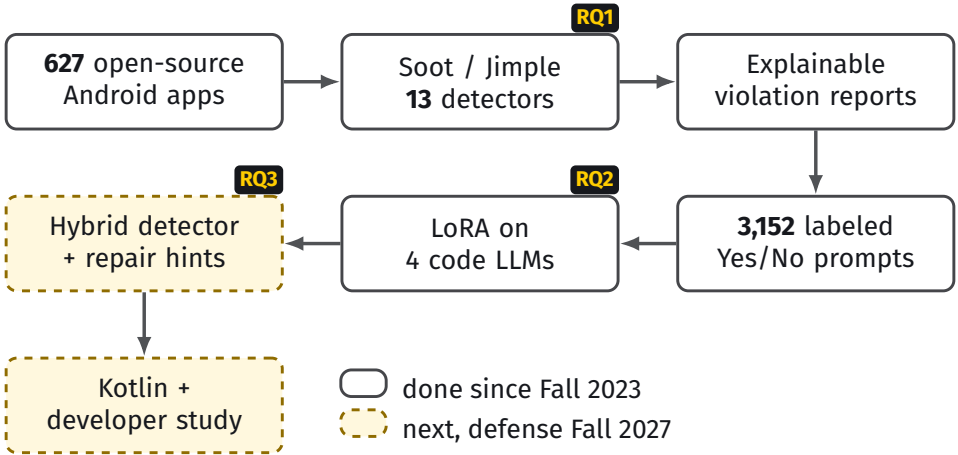
Hybrid detector

Three research questions

RQ1	Can Android documentation become **explainable static rules**?	**done**
RQ2	Does **parameter-efficient fine-tuning** make local code models useful?	**done**
RQ3	Does **fusing** rule evidence and model judgement beat either alone?	**next**

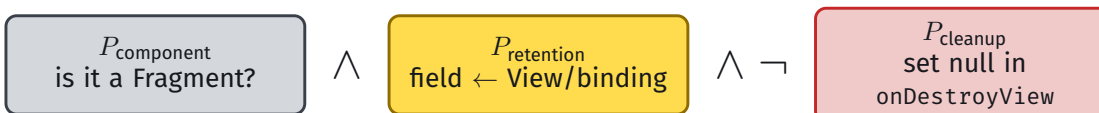
Accuracy · generalization · developer usefulness

Two stages done, one ahead



Android docs already state the rule

> “*Fragments* outlive their views. Make sure you clean up any *references to the binding* class instance in the fragment’s `onDestroyView()` method.”
>
> — Android Developers, *View binding* guide



Every detector is one such predicate cascade.

Rules run on Soot's Jimple IR

Java

```
view = inf.inflate(...);
```

⇒

Jimple (simplified)

```
$r2 = virtualinvoke r1.<...inflate(...)>
r0.<MusicianFragment: View view> = $r2
```

Pass 1
tag locals from `inflate`, `findViewById`, `bind` (+ casts)

→

Pass 2
field store from a tagged local ⇒ *retained*

Temporaries split one Java statement in two; the two passes reconnect them.

Each predicate is a few lines of code


```
if (!isFragment(cls)) return "";                                  P_component
// pass 1: locals produced by inflate / findViewById / bind
if (inViewLifecycle && isViewCreation(rhs)) viewLocals.add(lhs);
// pass 2: this.field = <view local>                              P_retention
if (isViewOrBindingType(field.getType()) && isViewValue)
    retained.put(field, methodName);
// this.field = null inside onDestroyView                         ¬P_cleanup
if (isDestroyView && rhs instanceof NullConstant) cleared.add(field);
retained.keySet().removeAll(cleared);   // remainder => report
```

Condensed from `FragmentViewFieldRetentionLeakDetector.java`.

Every warning explains itself


```
Fragment View Field Retention Leak
Class:   ...quanlyamnhac.fragment.MusicianFragment
Issue:   onDestroyView() is missing
Leaked:  view : android.view.View
         (assigned in onCreateView)
Fix:     onDestroyView() {
           super.onDestroyView(); view = null; }
```

what retained

where assigned

why no cleanup

how to fix

Real output on an open-source app. This text later becomes model supervision (RQ2).

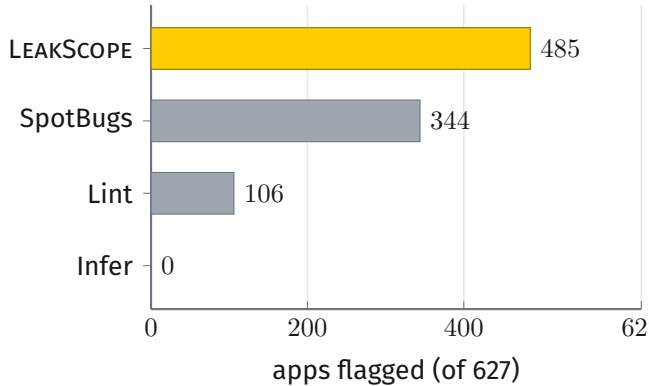
13 detectors behind one plug-in interface


```
interface ViolationDetector {
  String getName();
  String detectViolations(
      SootClass sootClass);
}
```

Registered via Java `ServiceLoader`: a new rule is one class.

Owner	Detectors
Fragment / View	FragmentViewFieldRetention, ViewBindingOpportunity, AndroidViewLifecycle
Threads	ThreadedUIReference, StateHolderLeak
ViewModel / async	ViewModelContextRef, FlowRetention
Service / Receiver	ServiceResourceMgmt, BroadcastReceiverBackgroundTask
Callbacks / resources	ImproperCallbackReg, BillingClientLeak, LoaderResetCursor, UnclosedCollator

Where others fire, LeakScope agrees

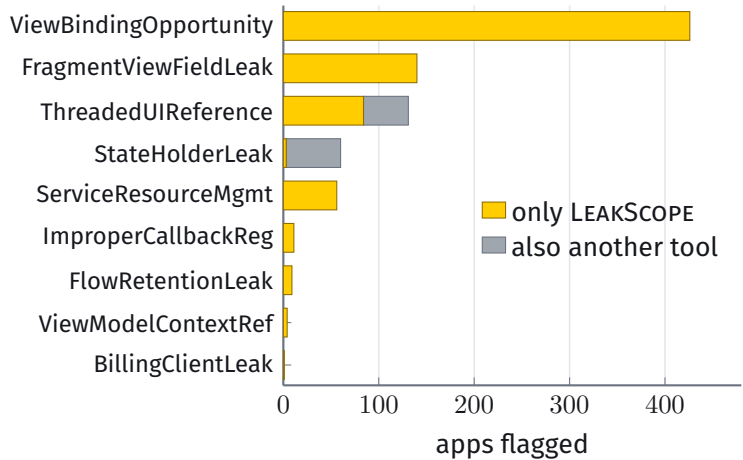


94%
of Lint's 106 apps also flagged by LeakScope (100)

85%
of SpotBugs' 344 apps also flagged (293)

Agreement ≠ ground truth: a 396-warning manual-validation sample is prepared.

734 findings no other tool reports



734

app–category pairs
no other tool reports

7 of 9

categories with zero
overlap

Gap: two patterns only Lint catches

static Context in an Application subclass

```
class App extends Application {
  private static Context context;
}
```

Our rule scans Activity / Fragment only.

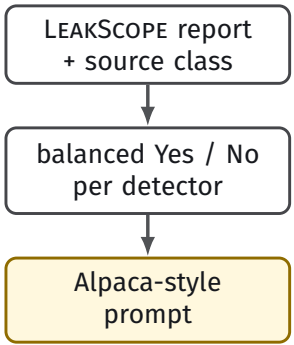
anonymous Handler captures its Activity

```
Handler h = new Handler(looper) {
  void handleMessage(Message m) {...}
};
```

Implicit outer-class reference.

StateHolderLeak: **Lint 70** apps vs. **LeakScope 60**, only 17 in common ⇒ extending rule scope is planned work

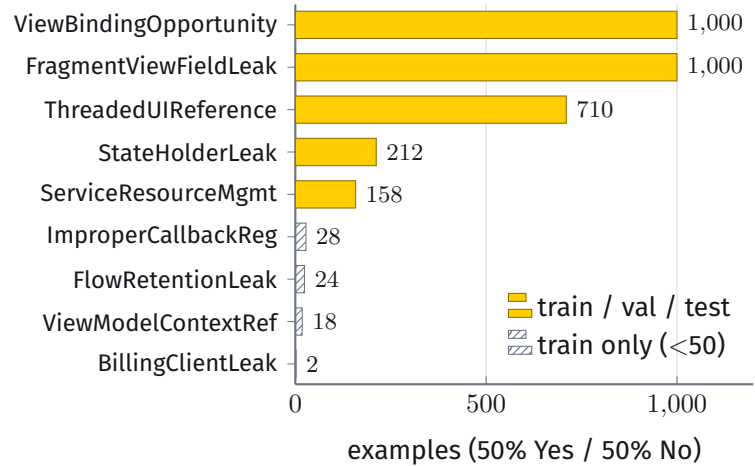
Detector reports become training data




```
### Instruction:
Determine whether this Android Java code contains a
ThreadedUIReference violation (worker thread captures
an Activity, Fragment, or View).
### Input:
<Java source, ≤ 2,500 chars>
### Response:
Yes. Captured UI objects: r0 : ...StackFragment
Fix: use WeakReference or avoid passing UI objects
```

Answer + static evidence: the model learns *why*, not just the label.

Data is long-tailed across detectors



2,535 / 307 / 310
train / val / test,
stratified per detector

5 of 9
detectors have enough data
to evaluate

Six systems, one held-out test set

Models

CodeLlama-7B
CodeLlama-13B
Llama-3-8B
StarCoder2-7B

GPT-4o (zero-shot ref.)
TF-IDF + LinearSVC

LoRA

rank 8 on `q_proj` / `v_proj`
3 epochs
seq. length 1,024 / 2,048

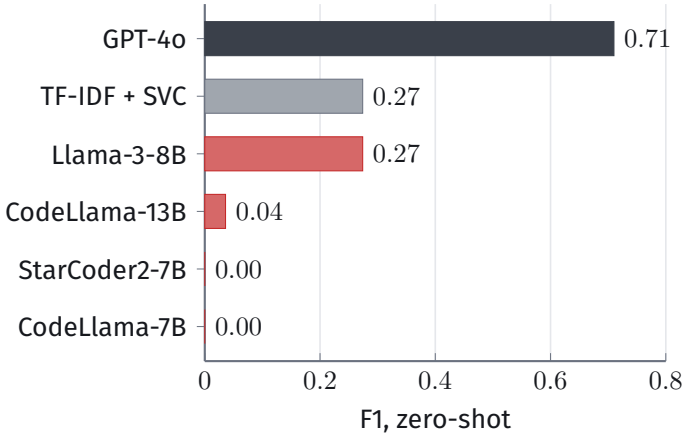
SDSC Expanse GPUs
local, deployable adapters

Test split

310 examples
160 violations
150 clean
5 detectors

precision, recall, F1
per detector + overall

Zero-shot open models just say “No”



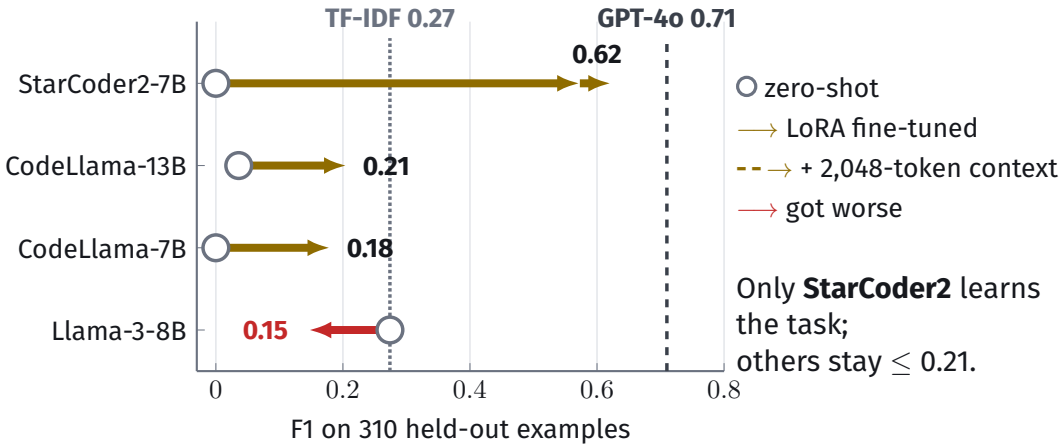
0.484

accuracy of CodeLlama-7B and StarCoder2-7B

= 150/310: *“No” to everything*

General code knowledge $\neq$ **lifecycle knowledge**

LoRA lifts StarCoder2 from F1 0.00 to 0.62



The fine-tuned model over-flags

StarCoder2-7B + LoRA

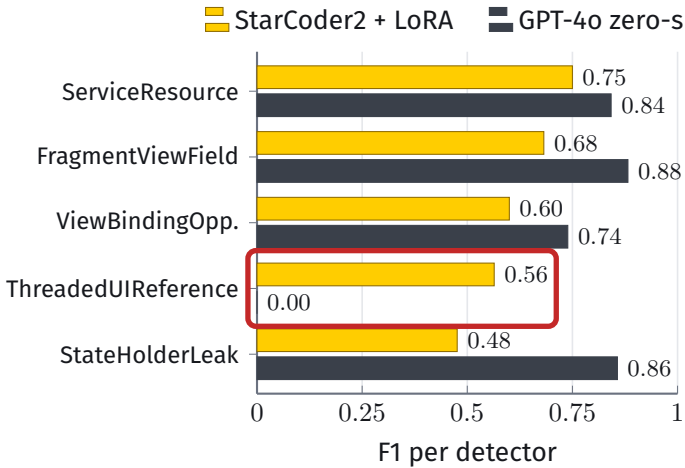
	pred. Yes	pred. No
violation	122	38
clean	110	40

GPT-4o zero-shot

	pred. Yes	pred. No
violation	103	57
clean	27	123

110 false alarms on 150 clean examples $\Rightarrow$ the fine-tuned model is a *recall* engine, not a judge

GPT-4o misses every thread-capture bug

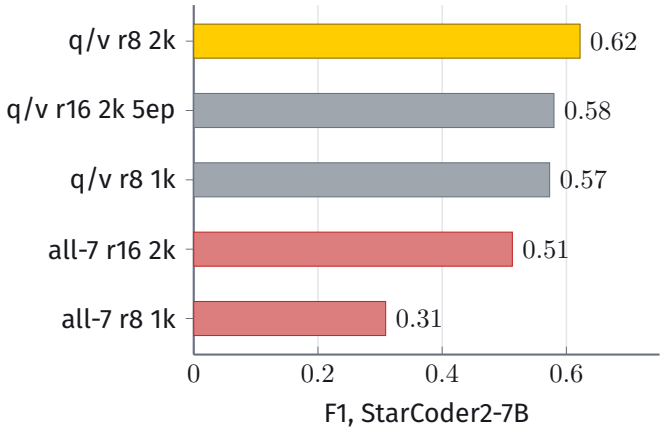


"The snippet primarily consists of import statements…"

— GPT-4o, on a true violation

- Truncated inputs hide the capture site
- Fine-tuned model still finds **22 / 37**

Longer context helps; wider LoRA hurts

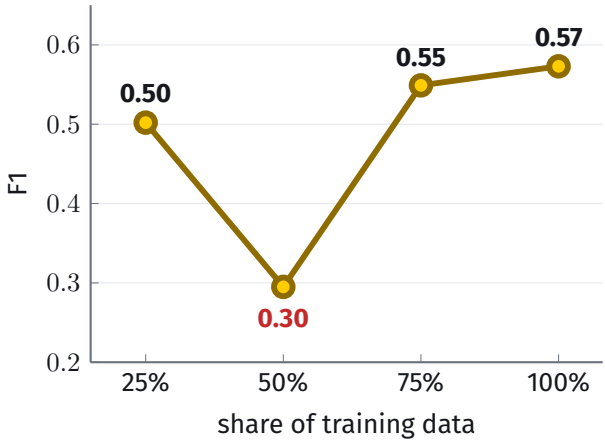


1,024 → 2,048 tokens
+0.049 F1

q/v → all 7 modules
−0.264 F1 (seq 1,024)

rank 16, 5 epochs
no gain over rank 8

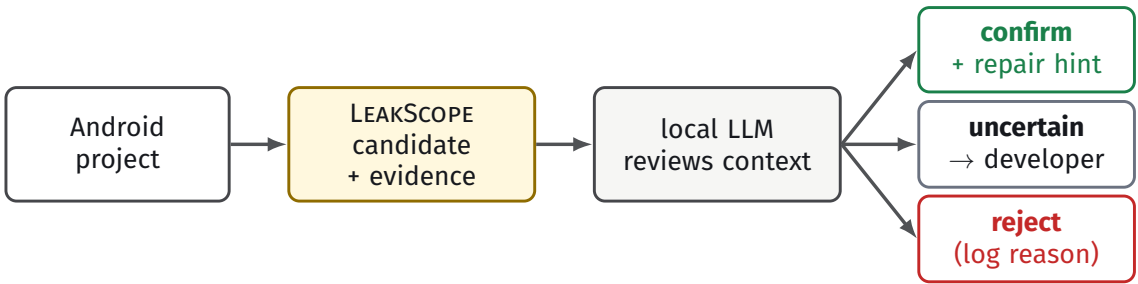
More data is not always better



- 50% subset **collapses** to 0.30
- Rare detectors: $\leq$ 28 examples each
- Sampling, not volume, is the lever

StarCoder2-7B, q/v r8, seq 1,024.

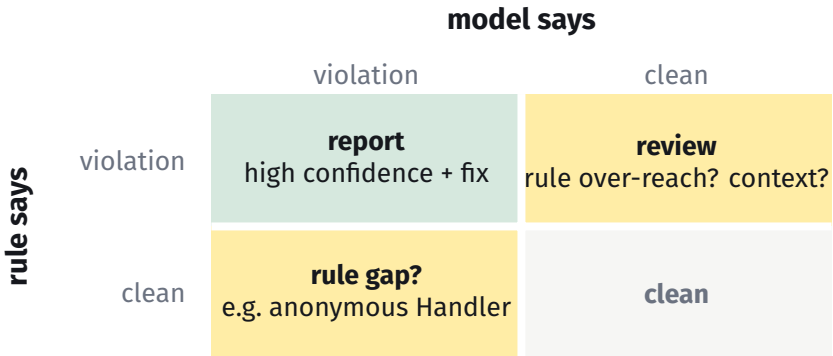
Next: rules propose, models review



Rule
traceable evidence, explicit scope

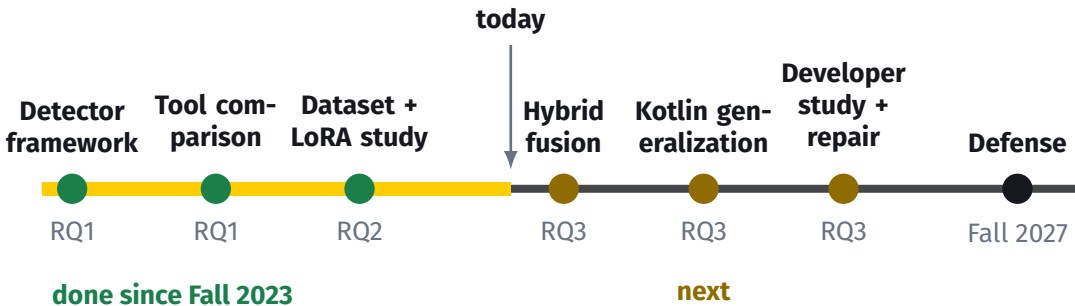
Model
context to cut the 110 false alarms

Disagreement is the richest signal



Off-diagonal cells feed back into new rules and new training data.

Path to defense in Fall 2027



RQ1 and RQ2 results feed directly into the RQ3 fusion design.

Where I would value your feedback

1. **Fusion policy:** trust the rule, the model, or escalate when they disagree?
2. **Ground truth:** best protocol to validate warnings at scale beyond two annotators?
3. **Kotlin:** coroutines, Flows, lambdas — extend Jimple rules or rely on the model?
4. **Developer study:** how to measure whether repair hints are *trusted*?

Rules find, models judge, fusion decides

RQ1

734

app–category hits no other tool reports

RQ2

0.00 → 0.62

F1 with LoRA; GPT-4o 0.71 but blind on threads

RQ3

Fuse

each fails differently, so combine them

Questions?

mxalitashfeen@shockers.wichita.edu

Thank you

This work would not exist without them.

ADVISOR

Dr. Zhiyong Shan
School of Computing
Wichita State University

CO-AUTHORS

Atish Shah Rouniyar
HomeServe USA Corp.

Muhammad Faraz Shoaib
Wichita State University

Hoang Thao Uyen Pham
Wichita State University

FUNDING

NSF
CNS-2154483
CNS-2452674

COMPUTE

SDSC Expanse